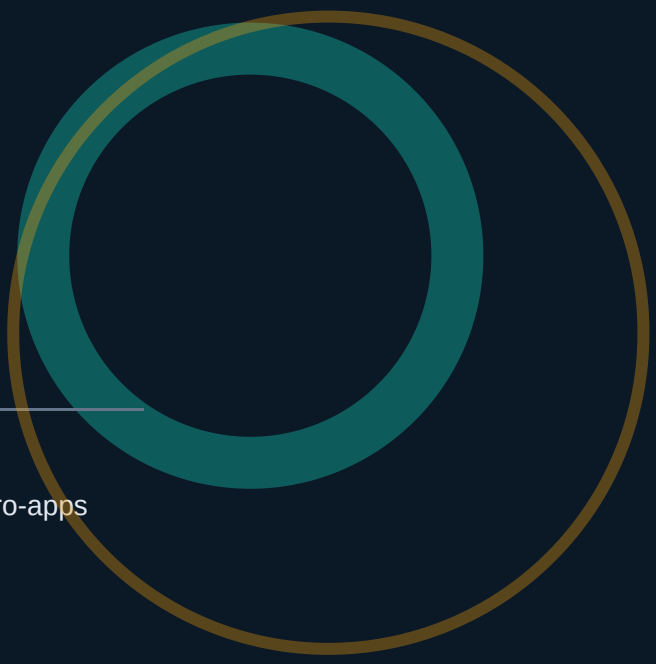
RUTA PERSONAL DE APRENDIZAJE

Full Stack de cero a producción

48 semanas para pasar del “vibe coding” al dominio consciente de HTML, CSS, JavaScript, TypeScript, React, Node.js, PostgreSQL, Supabase y Vercel.



Alejandro Osorio Loaiza (Alejo)
Enfoque: herramientas educativas, gestión de datos y micro-apps
Ritmo base: 10-12 horas por semana, ajustable

PUNTO DE PARTIDA

La meta no es escribir más código: es comprenderlo, probarlo y operarlo

Esta ruta aprovecha tu experiencia construyendo con IA, pero cambia el centro de gravedad: primero el modelo mental, luego la implementación y al final la asistencia.

Ya traes ventaja

- Problemas reales del sector educativo.
- Experiencia con HTML/JS, GitHub Pages y Google Sheets.
- Contacto con Supabase, PostgreSQL y Vercel.
- Capacidad para validar micro-apps con usuarios.

Brechas que vamos a cerrar

- Leer código sin depender de la explicación de una IA.
- Diseñar datos y APIs antes de generar pantallas.
- Depurar por evidencia, no por ensayo y error.
- Probar seguridad, permisos y fallos antes de publicar.

Resultado al terminar. Podrás diseñar, construir, desplegar y mantener una aplicación Full Stack; explicar sus decisiones; revisar código generado; y saber cuándo conviene un stack ligero o una arquitectura con base de datos, autenticación y backend.

Competencias de salida

Frontend
HTML semántico, CSS responsive, JavaScript, TypeScript, React, accesibilidad y pruebas.

Backend
HTTP, APIs REST, Node.js, validación, autenticación, autorización y logs.

Datos
Modelado relacional, SQL, PostgreSQL, migraciones, índices y Row Level Security.

Entrega
Git, ramas, revisión, CI básica, Vercel, variables por entorno y rollback.

Producto
Alcance, historias, métricas, feedback, privacidad y mantenimiento.

IA con control
Prompts acotados, revisión por diff, pruebas propias y explicación sin asistencia.

MAPA GENERAL

Ocho fases, una dependencia clara

No avances porque “ya viste el tema”. Avanza cuando puedes producir la evidencia de salida de la fase sin copiar una solución.

1. 1-6 Fundamentos de la web y control del entorno

2. 7-12 CSS, diseño adaptable y experiencia de usuario

3. 13-18 JavaScript de verdad: lenguaje, datos y lógica

4. 19-24 Navegador, APIs y aplicaciones con datos

5. 25-30 SQL y PostgreSQL: el corazón de los datos

6. 31-36 Backend con Node.js y APIs mantenibles

7. 37-42 React y TypeScript para interfaces escalables

8. 43-48 Supabase, Vercel y operación en producción

Fase	Semanas	Enfoque	Entrega principal
1	1-6	Fundamentos de la web y control del entorno	Guía web de recolección de evidencias para docentes, con formulario demostrativo y datos ficticios.
2	7-12	CSS, diseño adaptable y experiencia de usuario	Micrositio responsive para presentar un programa educativo y descargar una guía; sin framework.
3	13-18	JavaScript de verdad: lenguaje, datos y lógica	Motor de indicadores que recibe registros ficticios y genera resúmenes, alertas y exportación JSON.
4	19-24	Navegador, APIs y aplicaciones con datos	Explorador de avances con filtros y gráficas simples, alimentado por datos ficticios en JSON o una hoja pública de prueba.
5	25-30	SQL y PostgreSQL: el corazón de los datos	Base PostgreSQL para proyectos, sedes, docentes, actividades, evidencias e indicadores; siempre con datos sintéticos.
6	31-36	Backend con Node.js y APIs mantenibles	API de seguimiento de actividades educativas con roles coordinador/docente y documentación de endpoints.
7	37-42	React y TypeScript para interfaces escalables	Panel React + TypeScript para consultar indicadores, revisar evidencias y gestionar estados de una actividad.
8	43-48	Supabase, Vercel y operación en producción	Capstone: plataforma de seguimiento y evidencias o micro-app comercial con usuarios, permisos y operación documentada.

Ritmo estándar: 10-12 h por semana. Si el trabajo aprieta, conserva 5 h y mueve el entregable.

Regla de repetición: si falla el criterio de salida, repite con un ejercicio nuevo. No comprimas dos semanas.

Hitos: semana 12, sitio responsive; semana 24, app JavaScript con datos; semana 36, API con PostgreSQL; semana 48, capstone en producción.

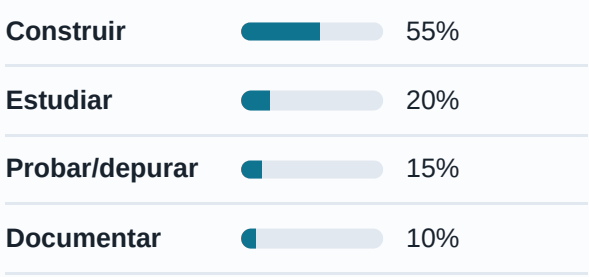
SISTEMA DE ESTUDIO

Dos horas diarias funcionan si cada sesión deja una evidencia

Usa seis días de trabajo y uno de descanso o repaso. El bloque del fin de semana absorbe integración, documentación y retrasos.

Día	Bloque de 2 horas	Evidencia
Lunes	25 min concepto · 70 min práctica · 25 min notas	Mapa mental y 3 preguntas.
Martes	20 min repaso · 90 min ejercicios · 10 min commit	Ejercicios resueltos sin copiar.
Miércoles	15 min predicción · 90 min construcción · 15 min explicación	Funcionalidad pequeña demostrable.
Jueves	20 min lectura de documentación · 80 min aplicación · 20 min pruebas	Pruebas y referencias anotadas.
Viernes	30 min depuración · 60 min refactor · 30 min README	Diff limpio y decisiones registradas.
Fin de semana	2-4 h de integración, demo y retrospectiva	Release semanal + retro de 5 líneas.

Distribución del tiempo



Retrospectiva semanal

1. ¿Qué puedo explicar sin abrir el código?
2. ¿Qué error entiendo ahora?
3. ¿Qué generó la IA que tuve que corregir?
4. ¿Qué prueba evita que el error vuelva?
5. ¿Cuál es el riesgo principal de la próxima semana?

> Tu unidad de progreso no es “horas vistas”; es una habilidad demostrada con código, pruebas y explicación.

PREPARACIÓN

Un entorno simple y reproducible evita aprender sobre arena movediza

Herramientas base

- Editor con terminal integrada.
- Git y una cuenta de GitHub.
- Navegador con DevTools.
- Node.js en versión LTS y un solo gestor de paquetes.
- PostgreSQL local o contenedor cuando llegues a la fase 5.
- Cliente de API y herramienta de diagramas.

Estructura de cada repositorio

```
proyecto/
  src/              código de la aplicación
  tests/            pruebas automatizadas
  docs/             decisiones y diagramas
  .env.example      nombres, nunca secretos
  README.md         problema, instalación,
uso
  CHANGELOG.md      cambios por versión
```

Configuración mínima

1. Nombre y correo de Git verificados.
2. Ramas protegidas en proyectos importantes.
3. Formato y lint ejecutables por comando.
4. Scripts de desarrollo, prueba y build.
5. Archivo de ejemplo de variables.
6. Datos ficticios para desarrollo.

> **Privacidad desde el día 1.** No uses nombres, documentos, teléfonos, correos, fotografías ni resultados reales de estudiantes en repositorios de práctica. Trabaja con datos sintéticos hasta tener autorización y controles adecuados.

Convención de commits

```
feat: agrega filtro por sede
fix: controla respuesta vacía
refactor: separa cálculo de indicador
test: cubre acceso de rol docente
docs: explica variables del entorno
```

1

SEMANAS 1-6

Fundamentos de la web y control del entorno

Entender qué ocurre entre un archivo, el navegador, el servidor y el repositorio; publicar una web sin depender de una plantilla.

Competencias

- Terminal básica y sistema de archivos
- Git: commits, ramas, merge y recuperación
- HTML semántico, formularios y accesibilidad inicial
- HTTP, DNS y modelo cliente-servidor a nivel conceptual

Semana	Tema	Qué dominar	Evidencia
1	Entorno y terminal	Rutas, archivos, procesos, editor, DevTools.	Repositorio de ejercicios con README.
2	Cómo funciona la web	URL, DNS, HTTP, petición/respuesta, navegador.	Diagrama explicado con tus palabras.
3	HTML semántico	Estructura, texto, enlaces, tablas, multimedia.	Página informativa sin CSS.
4	Formularios	Inputs, labels, validación nativa, errores.	Formulario accesible de evidencias.
5	Git y GitHub	status, add, commit, log, branch, merge, pull request.	Historial limpio con 8-12 commits.
6	Publicación	GitHub Pages, rutas, consola y depuración.	Sitio publicado y revisión móvil.

Proyecto de fase

Guía web de recolección de evidencias para docentes, con formulario demostrativo y datos ficticios.

CRITERIO DE SALIDA

Puedes crear, versionar y publicar una web semántica desde cero, explicar cada archivo y recuperar un error sencillo con Git.

2

SEMANAS 7-12

CSS, diseño adaptable y experiencia de usuario

Convertir una estructura correcta en una interfaz usable, accesible y adaptable sin copiar estilos que no comprendes.

Competencias

- Cascada, herencia, especificidad y box model
- Flexbox, Grid y diseño responsive
- Variables, componentes visuales y estados
- Accesibilidad, rendimiento visual y DevTools

Semana	Tema	Qué dominar	Evidencia
7	CSS esencial	Selectores, cascada, box model, unidades.	Hoja de estilos comentada.
8	Layout	Flexbox y Grid; alineación y distribución.	Dos layouts reconstruidos sin tutorial.
9	Responsive	Mobile-first, media queries, imágenes fluidas.	Tres anchos sin desbordamiento.
10	Sistema visual	Variables, tipografía, color, espaciado, componentes.	Mini sistema de diseño reutilizable.
11	Accesibilidad	Teclado, foco, contraste, jerarquía, mensajes.	Auditoría manual y correcciones.
12	Calidad visual	DevTools, rendimiento, impresión y QA.	Checklist visual antes/después.

Proyecto de fase

Micrositio responsive para presentar un programa educativo y descargar una guía; sin framework.

CRITERIO DE SALIDA

Reproduces un diseño desde una referencia, justificas cada regla importante y verificas móvil, teclado y contraste.

3

SEMANAS 13-18

JavaScript de verdad: lenguaje, datos y lógica

Dejar de ensamblar fragmentos y dominar las estructuras que permiten leer, modificar y depurar cualquier programa JavaScript pequeño.

Competencias

- Tipos, coerción, operadores y control de flujo
- Funciones, alcance, closures y módulos
- Arrays, objetos, transformaciones y algoritmos
- Errores, depuración y pruebas unitarias básicas

Semana	Tema	Qué dominar	Evidencia
13	Tipos y flujo	Valores, comparación, condicionales, bucles.	30 ejercicios breves razonados.
14	Funciones	Parámetros, retorno, scope, funciones puras.	Biblioteca de utilidades.
15	Colecciones	Arrays, objetos, map/filter/reduce.	Procesador de registros ficticios.
16	Módulos	import/export, separación de responsabilidades.	Aplicación en 4-6 módulos.
17	Asincronía	Promises, async/await, event loop conceptual.	Tres flujos con errores controlados.
18	Pruebas y debug	Casos, assertions, breakpoints, stack traces.	Suite mínima y bitácora de fallos.

Proyecto de fase

Motor de indicadores que recibe registros ficticios y genera resúmenes, alertas y exportación JSON.

CRITERIO DE SALIDA

Resuelves lógica sin IA, predices la salida antes de ejecutar y puedes explicar scope, mutación, asincronía y manejo de errores.

4

SEMANAS 19-24

Navegador, APIs y aplicaciones con datos

Construir una aplicación interactiva que consume datos, maneja estados y falla de forma comprensible.

Competencias

- DOM, eventos y formularios controlados
- Fetch, JSON, REST y códigos HTTP
- Estados loading/empty/error/success
- Persistencia local, validación y seguridad del cliente

Semana	Tema	Qué dominar	Evidencia
19	DOM y eventos	Selección, creación, delegación, formularios.	CRUD en memoria.
20	HTTP y Fetch	Métodos, headers, status, JSON, CORS.	Cliente de una API pública.
21	Estado de interfaz	Carga, vacío, error, éxito, reintento.	Máquina de estados simple.
22	Persistencia local	localStorage, serialización, migración simple.	Datos que sobreviven recarga.
23	Google Sheets	Lectura controlada, normalización, límites y caché.	Adaptador de fuente intercambiable.
24	Integración y QA	Validación, XSS conceptual, pruebas de flujo.	Versión publicada y casos de prueba.

Proyecto de fase

Explorador de avances con filtros y gráficas simples, alimentado por datos ficticios en JSON o una hoja pública de prueba.

CRITERIO DE SALIDA

Tu app funciona con red lenta, datos vacíos y errores; no incrustas secretos ni confías en validación solo del navegador.

5

SEMANAS 25-30

SQL y PostgreSQL: el corazón de los datos

Modelar información real, consultar con precisión y proteger la integridad antes de conectar una interfaz.

Competencias

- Modelo relacional, claves y normalización
- CRUD, joins, agregaciones, CTE y ventanas iniciales
- Restricciones, transacciones, índices y planes
- Migraciones, seeds, respaldo y permisos

Semana	Tema	Qué dominar	Evidencia
25	Modelo relacional	Entidades, relaciones, PK/FK, cardinalidad.	DER de un sistema educativo.
26	SQL básico	SELECT, INSERT, UPDATE, DELETE, filtros.	Archivo de 25 consultas.
27	Consultas útiles	JOIN, GROUP BY, HAVING, subconsultas.	Reporte reproducible.
28	Integridad	Constraints, transacciones, rollback.	Casos que demuestran protección.
29	Rendimiento	Índices, EXPLAIN conceptual, paginación.	Comparación de dos consultas.
30	Ciclo de esquema	Migraciones, seed, vistas, backup lógico.	Base recreable desde cero.

Proyecto de fase

Base PostgreSQL para proyectos, sedes, docentes, actividades, evidencias e indicadores; siempre con datos sintéticos.

CRITERIO DE SALIDA

Puedes diseñar un esquema, justificar claves y restricciones, escribir joins y reconstruir la base sin tocar manualmente producción.

6

SEMANAS 31-36

Backend con Node.js y APIs mantenibles

Comprender el servidor antes de delegarlo a una plataforma: entradas, reglas, datos, errores, autenticación y observabilidad.

Competencias

- Node.js, npm, variables de entorno y módulos
- Diseño REST, capas y validación del servidor
- PostgreSQL desde backend y transacciones
- Autenticación, autorización y pruebas de API

Semana	Tema	Qué dominar	Evidencia
31	Node y servidor HTTP	Runtime, npm, módulos, env, ciclo request.	Servidor con health check.
32	API REST	Recursos, rutas, status, errores consistentes.	CRUD versionado.
33	Arquitectura	Router, controller, service, repository.	Separación de responsabilidades.
34	PostgreSQL	Consultas parametrizadas y transacciones.	API conectada a esquema migrado.
35	Auth y permisos	Sesión/token conceptual, roles y ownership.	Matriz de autorización probada.
36	Pruebas y logs	Unitarias, integración, logs y manejo global.	Suite de API y diagnóstico.

Proyecto de fase

API de seguimiento de actividades educativas con roles coordinador/docente y documentación de endpoints.

CRITERIO DE SALIDA

Puedes seguir una petición completa, impedir acceso indebido, manejar errores sin filtrar detalles y probar rutas críticas.

7

SEMANAS 37-42

React y TypeScript para interfaces escalables

Pasar de manipulación manual del DOM a una interfaz por componentes, tipada y verificable sin ocultar los fundamentos.

Competencias

- Componentes, props, estado y renderizado
- Formularios, efectos, hooks y composición
- TypeScript: tipos, unions, interfaces, narrowing
- Routing, consumo de API y pruebas de UI

Semana	Tema	Qué dominar	Evidencia
37	TypeScript base	Tipos, funciones, objetos, unions, strict.	Migración de utilidades JS.
38	React mental model	Componentes, props, state, render.	Interfaz dividida por responsabilidades.
39	Interacciones	Eventos, formularios, estado derivado.	Formulario con validación visible.
40	Datos remotos	Fetch, efectos, cancelación, estados.	Vista resiliente a fallos.
41	Routing y diseño	Rutas, layouts, componentes y accesibilidad.	Flujo de 3-5 pantallas.
42	Pruebas y calidad	Pruebas de comportamiento, lint, build.	Suite de rutas críticas.

Proyecto de fase

Panel React + TypeScript para consultar indicadores, revisar evidencias y gestionar estados de una actividad.

CRITERIO DE SALIDA

Puedes explicar cada render, distinguir estado de datos derivados, tipar límites externos y probar comportamiento visible.

8

SEMANAS 43-48

Supabase, Vercel y operación en producción

Integrar frontend, base de datos, autenticación y despliegue con controles de seguridad, calidad y mantenimiento.

Competencias

- Supabase Auth, Storage, SQL y Row Level Security
- Migraciones y separación dev/preview/production
- Vercel, variables, builds, previews y dominio
- Monitoreo, rendimiento, seguridad y entrega

Semana	Tema	Qué dominar	Evidencia
43	Supabase local/proyecto	Esquema, migraciones, seed y cliente.	Entorno reproducible.
44	Auth y RLS	Usuarios, claims, policies por rol/propiedad.	Pruebas positivas y negativas.
45	Storage	Buckets, rutas, tamaños, tipos y permisos.	Carga segura de evidencia ficticia.
46	Integración completa	React + Supabase; errores y caché.	Flujo de punta a punta.
47	Vercel y CI	Preview, variables por entorno, build, rollback.	Release candidate validado.
48	Producción	Auditoría, métricas, backups, runbook y demo.	Capstone publicado y documentado.

Proyecto de fase

Capstone: plataforma de seguimiento y evidencias o micro-app comercial con usuarios, permisos y operación documentada.

CRITERIO DE SALIDA

Despliegas con migraciones, políticas RLS verificadas, secretos fuera del repositorio, checklist de release y plan de recuperación.

HITO INTERMEDIO

Al final de la semana 24 debes poder construir sin framework

Este checkpoint confirma que React no ocultará vacíos en HTML, CSS, JavaScript, DOM, HTTP o manejo de errores.

Reto de 6 horas

Construye desde un repositorio vacío un explorador de datos educativos ficticios.

1. Carga un JSON local o endpoint permitido.
2. Muestra carga, vacío, error y éxito.
3. Filtra por sede y periodo.
4. Calcula al menos dos indicadores.
5. Permite exportar el resultado.
6. Publica y documenta decisiones.

Sin estas ayudas

- Sin copiar un tutorial completo.
- Sin framework de interfaz.
- Sin pedir a la IA “haz toda la app”.
- Sin secretos ni datos personales.
- Sin ignorar errores de consola.

Se permite

Documentación oficial, búsquedas puntuales, autocompletado y preguntas acotadas después de intentar y registrar el fallo.

Lista de aprobación

Área	Prueba	Aprobación
Estructura	HTML semántico y navegación por teclado.	Sin bloqueos ni controles sin etiqueta.
Diseño	320 px, 768 px y escritorio.	Sin desbordamientos ni texto ilegible.
JavaScript	Módulos y funciones pequeñas.	Puedes explicar el flujo de datos.
Red	Simula lento, offline y respuesta vacía.	La interfaz comunica cada estado.
Calidad	Pruebas de cálculos y validaciones.	Los casos críticos quedan automatizados.
Entrega	README reproducible y commits claros.	Otra persona puede ejecutarlo.

ESCALERA DE PROYECTOS

Cada proyecto reutiliza el anterior y añade una capa de responsabilidad

1 · Guía de evidencias

Fases 1-2. Sitio estático, accesible y responsive. Publicado en GitHub Pages.

Demuestra: HTML, CSS, formularios, Git, documentación y QA visual.

2 · Explorador de indicadores

Fases 3-4. Aplicación JavaScript con filtros, cálculos, estados y exportación.

Demuestra: lógica, DOM, módulos, Fetch, errores y pruebas.

3 · API de seguimiento

Fases 5-6. PostgreSQL + Node.js con roles, validación y pruebas de integración.

Demuestra: SQL, arquitectura, autorización y observabilidad.

4 · Capstone Full Stack

Fases 7-8. React + TypeScript + Supabase + Vercel, con migraciones y Row Level Security.

Demuestra: producto completo, seguridad, despliegue y operación.

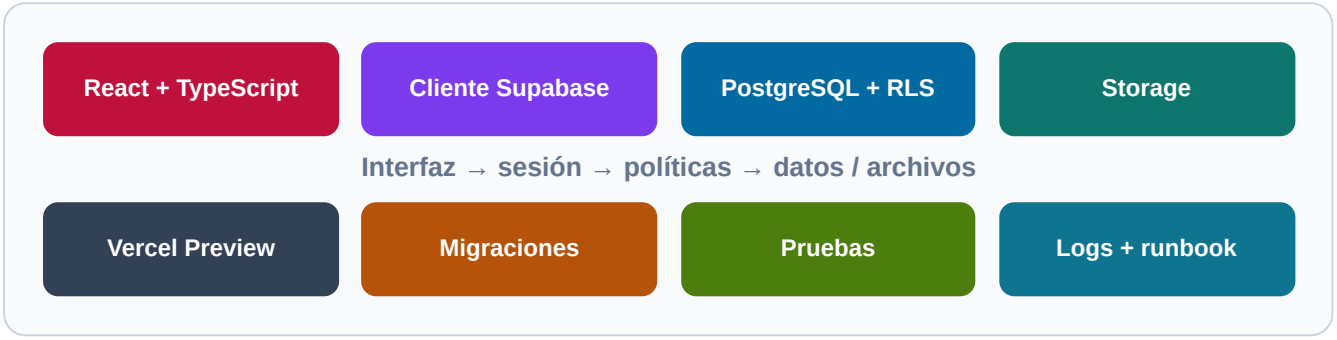
Tres opciones de capstone

Opción	Usuario y problema	MVP	Riesgo a controlar
Gestor de evidencias	Docentes cargan y clasifican evidencias; coordinadores revisan avance.	Actividades, carga, estado, comentarios y resumen.	Permisos de archivos y datos personales.
Portal de informes	Aliados consultan indicadores y descargan reportes autorizados.	Filtros, vistas guardadas, exportación y auditoría.	Filtración entre organizaciones.
Micro-app comercial	Un nicho registra procesos repetitivos y recibe alertas.	Onboarding, CRUD, roles, límites y panel.	Alcance excesivo y falta de validación.

Regla de producto. Antes de programar el capstone, entrevista o prueba el flujo con 3-5 usuarios potenciales usando un prototipo sin datos reales. Construye solo el problema que se repita.

ARQUITECTURA DEL CAPSTONE

Una arquitectura sencilla gana si puedes explicarla de punta a punta



Flujos mínimos

1. Registro/inicio de sesión.
2. Creación de una actividad.
3. Carga o asociación de evidencia.
4. Revisión según rol.
5. Consulta y filtro.
6. Error, reintento y recuperación.

Tablas orientativas

```
organizations
profiles
memberships
projects
activities
evidence_items
reviews
audit_events
```

Definition of Done del capstone

- Historias y fuera de alcance escritos.
- Diagrama de datos con claves y restricciones.
- Migraciones y seed reproducibles.
- RLS probada por rol y organización.
- Estados de interfaz completos.

- Pruebas de rutas críticas.
- Variables separadas por entorno.
- Preview revisada antes de producción.
- README, demo y registro de decisiones.
- Runbook de incidentes y rollback.

No uses un backend adicional por reflejo. Si Supabase cubre autenticación, datos, archivos y reglas con claridad, empieza allí. Añade Node.js cuando exista lógica de servidor, integración o control que lo justifique.

EVALUACIÓN

La competencia se demuestra en cinco dimensiones, no con una app que “se ve bien”

Dimensión	1 · Inicial	2 · Funcional	3 · Autónomo	4 · Profesional
Comprensión	Repite código.	Explica partes.	Predice cambios.	Justifica alternativas.
Implementación	Depende de guía.	Completa flujo feliz.	Maneja bordes.	Diseña para cambio.
Depuración	Prueba al azar.	Lee errores.	Aísla la causa.	Previene regresión.
Calidad	Sin pruebas.	Pruebas manuales.	Automatiza lo crítico.	Cubre riesgo y operación.
Entrega	Solo funciona local.	Publica con ayuda.	Reproduce y documenta.	Observa y recupera.

Umbral para avanzar. Obtén al menos nivel 3 en Comprensión y Depuración, y nivel 2 o superior en las demás dimensiones. Repite el reto con un contexto nuevo si solo funciona porque recuerdas el tutorial.

Examen de cierre de cada fase

Parte A · Construcción (60%)

- Reto nuevo con tiempo limitado.
- Commits pequeños y mensaje claro.
- Casos normales y fallos.
- README de ejecución.

Parte B · Defensa (40%)

- Explica el flujo sin leer.
- Cambia un requisito en vivo.
- Diagnostica un fallo introducido.
- Nombra un riesgo y su control.

Tablero personal

Semana	Entrega	Nivel	Fallo aprendido	Siguiente riesgo
__	_____	__/4	_____	_____
__	_____	__/4	_____	_____
__	_____	__/4	_____	_____

IA BAJO CONTROL

Usa la IA como revisor y tutor, no como autora invisible

Protocolo PACTO

1. **Problema:** escribe entrada, salida, restricciones y ejemplo.
2. **Anticipa:** predice el enfoque y los errores probables.
3. **Construye:** intenta 25-40 minutos sin solución completa.
4. **Trabaja con IA:** pide una pista, crítica o prueba concreta.
5. **Opera:** ejecuta, prueba, explica y registra la decisión.

Prompts que sí enseñan

- “No me des código. Hazme tres preguntas para encontrar el error”.
- “Revisa este diff y señala riesgos; no lo reescribas”.
- “Propón casos de prueba que contradigan mis supuestos”.
- “Explícame el stack trace paso a paso y detente antes de la solución”.
- “Compara estas dos opciones con criterios explícitos”.

Semáforo de uso

Verde	Amarillo	Rojo
Explicaciones, preguntas socráticas, casos de prueba, revisión de diff, documentación puntual.	Boilerplate, refactor, SQL o políticas: solo con revisión línea por línea y pruebas negativas.	Aplicación completa, autenticación o migraciones generadas y desplegadas sin comprensión ni revisión.

> **Prueba de propiedad.** No hagas merge de código generado si no puedes: explicar su flujo, cambiarlo sin romperlo, probarlo y decir qué falla cuando la entrada es inválida.

Bitácora mínima de decisiones

```
Decisión: usar RLS por organization_id
Motivo: impedir lectura entre organizaciones
Alternativas: filtro en cliente; backend propio
Pruebas: usuario A no consulta filas de B
Riesgo restante: políticas nuevas sin cobertura
```

PRODUCCIÓN

Publicar es el comienzo de la responsabilidad técnica

Checklist de seguridad

- Secretos fuera del repositorio y rotables.
- Validación en servidor o base, no solo en UI.
- Autorización por recurso, rol y organización.
- RLS con pruebas positivas y negativas.
- Consultas parametrizadas.
- Archivos con tipo, tamaño y acceso limitados.
- Errores públicos sin detalles internos.
- Dependencias revisadas y actualizadas.

Checklist de operación

- Build y pruebas antes del despliegue.
- Preview revisada con datos ficticios.
- Migración reversible o plan de restauración.
- Logs suficientes sin datos sensibles.
- Health check y monitoreo básico.
- Backup y restauración ensayados.
- Responsable y canal de incidentes.
- Runbook: detectar, contener, recuperar, aprender.

Ladder de entornos

Entorno	Propósito	Datos	Regla
Local	Desarrollo rápido.	Sintéticos.	Puede borrarse y reconstruirse.
Preview	Validar cambios por rama.	Sintéticos o anonimizados autorizados.	No comparte secretos de producción.
Producción	Uso real.	Mínimos necesarios.	Cambios revisados, observables y recuperables.

Datos educativos. Aplica minimización, propósito definido, acceso por necesidad, retención limitada y trazabilidad. La tecnología no reemplaza la autorización institucional ni las políticas aplicables.

PORTAFOLIO

Tu repositorio debe mostrar cómo piensas, no solo capturas de pantalla

README que convence

1. Problema y usuario.
2. Alcance y fuera de alcance.
3. Arquitectura y decisiones.
4. Instalación reproducible.
5. Variables sin valores secretos.
6. Pruebas y cómo ejecutarlas.
7. Capturas o demo.
8. Limitaciones y próximos pasos.

Evidencia por proyecto

- Commits pequeños y comprensibles.
- Issues o tablero de trabajo.
- Pull request con auto-revisión.
- Diagrama actualizado.
- Casos de prueba críticos.
- Registro breve de decisiones.
- Release etiquetada.
- Retrospectiva honesta.

Rutina profesional mensual

Semana	Acción visible	Resultado
1	Mejora documentación de un proyecto.	Otra persona lo ejecuta sin ayuda.
2	Refactoriza una deuda con pruebas.	Diff pequeño y riesgo reducido.
3	Revisa código ajeno o un repositorio de referencia.	Notas de patrones y preguntas.
4	Publica demo y retrospectiva técnica.	Una pieza de portafolio verificable.

Repositorio de referencia del contexto laboral
https://github.com/alianzaeducacionrural
Úsalo para observar estructura, convenciones y necesidades reales; no para copiar sin comprender ni exponer información interna.

PRIMEROS 90 DÍAS

Empieza con fundamentos y una entrega real cada cuatro semanas

Días 1-30

Objetivo: entorno, web y Git.

- Completa semanas 1-4.
- Publica HTML semántico.
- Haz 20+ commits intencionales.
- Explica HTTP con un diagrama.

Días 31-60

Objetivo: publicación y CSS.

- Cierra semanas 5-8.
- Practica ramas y recuperación.
- Construye layouts sin tutorial.
- Revisa móvil y teclado.

Días 61-90

Objetivo: responsive y accesibilidad.

- Cierra semanas 9-12.
- Define sistema visual.
- Audita contraste y foco.
- Entrega proyecto de fase 2.

Semana 1, día por día

Día	Acción	Hecho cuando...
1	Instala/verifica editor, Git, terminal y navegador.	Guardas versiones y salidas en README.
2	Practica rutas, crear, mover, copiar y borrar archivos.	Repites sin interfaz gráfica.
3	Crea repositorio y primeros commits.	log cuenta una historia comprensible.
4	Abre DevTools: Elements, Console, Network.	Ubicas un recurso y su status.
5	Escribe HTML mínimo y publícalo.	La URL abre y no hay error de rutas.
6	Rompe algo a propósito y recupéralo con Git.	Documentas causa y recuperación.
7	Descanso o repaso de 30 minutos.	Explicas la semana sin abrir notas.

> **Tu única tarea inicial.** Crea un repositorio llamado de forma descriptiva, añade un README con el objetivo de la ruta y entrega la primera página HTML publicada. No instales React todavía.

REFERENCIAS

Documentación primaria primero; cursos y videos, después

Web y JavaScript

MDN · Aprende desarrollo web
http://developer.mozilla.org/es/docs/Learn_web_development

MDN · Módulos centrales
https://developer.mozilla.org/es/docs/Learn_web_development/Core

Git

Repositorio de Pro Git, segunda edición
https://github.com/progit/progit2

Repo de referencia del usuario
https://github.com/alianzaeducacionrural

Frontend tipado

Código y documentación de React
https://github.com/reactjs/react.dev/

Repositorio de TypeScript
https://github.com/can1357/typescript

Datos y plataforma

Supabase con React
https://supabase.com/docs/guides/getting-started/quickstarts/reactjs

Vercel Docs sitemap
https://vercel.com/docs/sitemap.md

Cómo usar los recursos

1. Lee solo el bloque que responde a la tarea de la semana.
2. Escribe una predicción antes de ejecutar el ejemplo.
3. Recrea el concepto en un contexto distinto.
4. Guarda una nota con “qué es”, “cuándo usarlo” y “cómo falla”.
5. Vuelve a la documentación cuando una dependencia cambie; no confíes en memoria de versiones.

> **Decisión recomendada.** Mantén esta ruta como plan maestro y crea un tablero con 48 filas: estado, evidencia, nivel de rúbrica, error aprendido y próxima acción.

Ruta adaptable: si una tecnología cambia, conserva la competencia y actualiza la herramienta. El objetivo es entender sistemas web, no coleccionar frameworks.